

erates at 3.3V power supply, it is necessary to use a proper level shifter/buffer in the SPI bus when it is interfaced with a microcontroller operating at 5V supply.

The LCD is operated in 4-line mode. Port pins PD4 through PD7 of IC1 are used as data lines. Pins PD1, PD2 and PB6 serve as control lines for the LCD display. Pin PD0 acts as chip select pin for IC2. Pins PB1 and PB2 are

connected to SCK and SI pin of IC2, respectively. Pin PB3 is connected to SO pin of IC2 via level shifter IC3 to maintain proper TTL level since IC2 is operated at a 3.3V power supply. The INT pin of IC2 is not used. We have used 74HCT125 as the level-shifter-interface IC between the microcontroller and the Ethernet controller. A 16MHz crystal is used as the external clock source for the microcontroller. A TYCO magnetic jack is used to establish connections between Ethernet controller IC2 and PC using RJ45 connectors.

The water-level sensor rods (A, B, C and D) are connected to the bases of four BC547 transistors whose emitters are connected to port pins PF0 through PF3 of IC1. The logic levels used for sensing water level in the overhead tank are shown in Table II.

Depending upon water level in the tank, a graphical image of the water tank is displayed on the main Web page. Here, the uniqueness is that the information is displayed as an image instead of data on the Web page. The water-level data is updated and displayed as a visual which is refreshed

using AJAX in the HTML code. Timer function is not included in the AJAX code, so the refreshing rate of the page depends upon the size of updated data. If the data contained no visual information, the refreshing rate would be faster.

Port pin PC1 of IC1 is used to drive the relay circuit. LED2 connected to pin PC1 simulates the motor on/off action by clicking 'motoron' and 'motoroff' buttons in the HTML page. The electric motor is turned on/off through the relay. Transistor T5 is used as a relay driver.

Software program

Software for the ATmega128 microcontroller is written in C.

SPI communication between ATmega128 and ENC28J60 is first initialised, as given below, with the selection of master and clock signals:

$SPCR = (1 < < SPE) | (1 < < MSTR);$

$SPSR = (1 < < SPI2X);$

All the instructions for SPI communication are available in ENC28J60.c source file included in this month's EFY DVD.

After SPI initialisation, ENC28J60 should be initialised for proper network protocol transmission. ENC28J60 initialisation and other relevant codes are also included in the ENC28J60.c file. The ENC28J60 uses a number of software routines for proper operation. These routines are taken from Microchip TCP/IP stack and ported to our application. MACinit () routine does necessary initialisation of registers in ENC28J60 module.

Note that the Ethernet controller should be first connected to ATmega128 via SPI bus and should be initialised by issuing MACinit () routine. Some important routines used in this project are mentioned here:

MACPutHeader () routine is used for preparing header for outgoing Ethernet packet

MACFlush () routine is used to send Ethernet packet

MACGetHeader () routine is used to analyse header in the incoming packets.

MACPut () routine helps to pre-



Fig. 3: ATmega128 module

TABLE II
Logic Levels Used to Sense Water Level

D	C	B	A	Status of the tank
0	0	0	0	Empty tank
0	0	0	1	Low or 25%
0	0	1	1	Half or 50%
0	1	1	1	Three-fourth or 75%
1	1	1	1	Full or 100% (all sensors inside water)

TABLE III
Description of Sub-Routines

Subroutine name	Description
Main(void)	After SPI initialisation, continuously poll for incoming messages by calling stack init())
MACInit(void)	Initialise ENC28J60 registers after power up
In/output(,)	Read/write word of data to/from ENC28J60 registers via SPI bus interface
Stack init()	Check ENC28J60 for any new incoming Ethernet messages. If there are any, process them
Reply_tcp(void)	Reply to a requested TCP/IP packet. Web requests are served with this packet
Decode_tcp(void)	Decode the type of the incoming TCP packet
Ne_send(,)	Send the already prepared Ethernet packet in the ENC28J60 SRAM buffer
MACPutHeader(,)	Fill in MAC/IP header portion of new outgoing packet in the ENC28J60 transmission buffer
Write_ip/tcp_check(,)	Read back contents of outgoing packet and append checksum in appropriate spot
Flush_print(void)	Data is written into the ENC28J60; one word at a time. This does clean up to flush any remaining byte
Ne_prints(,)	Write one byte of data into Ethernet packet, taking care of checksums, byte counts, etc
Finish_tcp(,)	After writing TCP packet, clean it up and send it